计算机系统基础 • BombLab 实验报告

Made by:陈南晖 25301020039

摘要

本实验以个人 ELF64 x86-64 程序 bomb 为分析对象，在只修改 defuser.c 的约束下，利用 objdump 静态反汇编、GDB 内存观察和运行测试，恢复各关卡从 64 位 challenge 生成口令的计算关系。普通六关依次涉及函数参数传递、递推数组、跳转表、递归回溯、链表内存结构、函数指针表；Honor 还要求综合处理 context、字符串缓冲区与固定内存偏移。调试中不仅完成算法复现，也记录了编译失败、执行旧二进制、输入格式假设错误等问题的定位和改正。最终公开测试全部通过，并对公开列表外的 seed 执行了完整验证。

一、实验目标、约束与方法

课程要求编写能够处理不同 challenge 的自动解题程序，而非写死某次运行的口令。普通关卡 solve_phase_1 至 solve_phase_6 通过 uint64_t challenge 计算 uint32_t 整数序列；Honor 的 solve_secret 额外接收 uint64_t context，并输出带文本结构的一行。编译使用 -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror，必须处理编译警告。

基本方法为：先定位关卡和辅助函数地址，再识别参数寄存器与比较逻辑；借助 C 语言等价伪代码推导计算关系；将算法写入 defuser.c；通过 make、单关管道测试、公开测试和非公开 seed 交叉验证。重点把静态分析与动态内存观察结合起来，避免把反汇编中的地址、数据和控制流程孤立理解。

二、环境搭建与前期排错

最初通过 SSH 访问实验服务器后，git clone 多次遇到 GnuTLS recv error (-110) 或连接 github.com:443 超时。curl 又提示 error (77)，进一步检查发现系统缺少 /etc/ssl/certs/ca-certificates.crt，dpkg 显示 ca-certificates 未安装。使用学校可访问的 Ubuntu 镜像运行 apt-get update、apt-get install -y ca-certificates 后，curl -I https://github.com 返回 HTTP/2 200；此后网络仍曾间歇失败，重新尝试才克隆成功。此过程表明证书信任链故障与网络传输超时应当区分处理。**最终确定为虚拟机网络连接不稳定导致的克隆失败。（希望助教能确保虚拟机与 GitHub 网络的稳定连接 QAQ）**

克隆后查看 README、Makefile 与 config.txt，填写领取个人 bomb 所需的学号，执行 make bomb 获取 bomb 与 package.json；make 成功编译 defuser，make tutor 编译教学关。教学关用 phase_00、tutorial_mix 练习了寄存器传参、lea 算术运算及将汇编转换为 C 表达式的方法。

三、各关卡关键汇编与恢复过程

3.1 Phase 1：函数调用与参数传递

phase_01 验证恰好一个无符号整数，调用 p1_compute(challenge) 并比较返回值。p1_compute 先调用 p1_step1，再调用 p1_step0，最后调用 p1_step2；通过 rdi、rsi、rax、rdx、rcx 等寄存器追踪参数与返回值，可还原调用顺序。

```
high = (uint32_t)(challenge >> 32);
middle = (uint32_t)(challenge >> 16);
low = (uint32_t)challenge;
t1 = (middle + 0xc699) ^ high;
t2 = (3 * low) ^ (high + 0x71b5);
answer = 5 * (t2 - t1) + 0x96fa;
```

关键指令如 lea (%rdi,%rdi,2),%eax 表示 3×rdi，而不是取内存地址；32 位寄存器赋值带来模 2^32 的运算效果。最初先比对辅助函数与 C 代码，再通过 seed 42 和 20261027 的单关测试及公开列表验证。

3.2 Phase 2：数组与递推式

phase_02 首先用 valid_u32_fields 确认输入含六个整数，然后将它们放在栈上的连续位置比较。第一项取 challenge 第 24 位开始的 5 位，再乘 3；后续五项通过循环生成。汇编的 -0x40(%rbp,%rdx,4) 对应第 i 个 32 位数组元素。

```
a[0] = 3 * ((challenge >> 24) & 0x1f);
for (uint32_t i = 1; i < 6; ++i) {
    uint32_t noise = (challenge >> (20 + i)) & 0xf;
    a[i] = 5 * (a[i-1] + i) - noise;
}
```

本关曾反复出现“solver is incomplete or failed”，其中一个重要排错点是编辑 defuser.c 后未立即重新 make，运行的仍是旧编译产物。重新编译后，seed 20261027 生成序列“30 150 750 3760 18810 94070”并成功解除，随后公开测试全部通过。

3.3 Phase 3：switch 与跳转表

phase_03 要求两个整数。第一个由 challenge & 7 决定；第二个基于 x = (uint32_t)((challenge >> 3) ^ (challenge >> 32))，送入 p3_dispatch。指令 notrack jmp *0x405820(,%rax,8) 表明按索引读取 8 字节目标地址并跳转。GDB 读取 0x405820 起始的八项，确认入口次序不等同于 case 函数在 .text 中的地址排序。

索引	跳转桩地址	对应运算
0	0x402258	case_5：5*x + 0xf0fc
1	0x402260	case_4：(x ^ 0x8a21) + (x >> 1)
2	0x402268	case_6：5*x + 0x3f408
3	0x402270	case_1：5*x + 0x2cc27
4	0x402278	case_3：(x ^ 0x6853) + (x >> 1)
5	0x402280	case_2：(4*x) ^ 0x4373
6	0x402248	case_7：(2*x) ^ 0x7bf3
7	0x402250	case_0：3*x + 0xd3c8

认识到“case 编号”和“跳转表索引”必须分开处理后，按实际地址恢复映射，seed 42、20261027 和全部公开 seed 均测试通过。

3.4 Phase 4：递归二分搜索与返回值回溯

phase_04 根据 challenge 取 target = challenge & 0x7f、salt = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32)，再调用 p4_walk(target, 0, 127, salt)。函数每次计算 mid = low + (high-low)/2，在中点命中时返回 (mid+0xd1)^salt；若继续向左或向右递归，则在返回时把结果乘三，与 mid+salt 异或，再异或分支常量（左 0xd，右 0x1）。

```
if (target == mid) return (mid + 0xd1) ^ salt;
key = mid + salt;
```

```
if (target < mid) return ((3 * walk(target,low,mid-1,salt)) ^ key) ^ 0xd;
return ((3 * walk(target,mid+1,high,salt)) ^ key) ^ 0x1;
```

分析过程中考虑过改为非递归循环。但本关的难点不仅是搜索路径，还包括递归返回阶段逐层加工结果；保留递归结构更易忠实对应原汇编。修改文件时曾误删 solve_secret 的函数头，使文件末尾出现孤立的 (void)challenge 和 return 语句，从而引起 expected identifier 报错；随后又误把 context 声明成 const char *，与 defuser.h 中 uint64_t 不一致。修复函数边界和签名后成功编译及通过测试。



图 1 Phase 4 调试早期的失败运行与函数结构语法错误



图 2 solve_secret 声明类型冲突及修复后 Phase 4 成功运行

3.5 Phase 5：链表结构恢复和数组重排

phase_05 解析两个整数，调用 p5_traverse(challenge, &tag)，比较遍历结果和最终 tag。p5_heads 位于 0x405b60，共八个头指针；p5_nodes 位于 0x405ba0，每个节点占 16 字节，布局为 uint32_t value（偏移 0）、uint32_t tag（偏移 4）、8 字节 next 指针（偏移 8）。例如 0x00000a5800000596 按小端序可解为 value=0x0596、tag=0x0a58；next 为 0x405c00。

观察到多个头指针指向同一条链的不同起始节点，而不是八条相互独立的链。通过按 next 指针追踪，恢复了从 0x405bd0 开始的十二节点顺序：0x405bd0 → 0x405c50 → 0x405ba0 → 0x405c00 → 0x405c30 → 0x405be0 → 0x405bb0 → 0x405bc0 → 0x405bf0 → 0x405c40 → 0x405c20 → 0x405c10。前八个头指针恰好对应上述链的下标 0—7（顺序对应 p5_heads 的八个入口）。

实现时不依赖原 bomb 运行时的绝对指针，而是将节点按逻辑连接顺序存成 P5Node {value, tag} 静态数组。由 index=challenge&7，steps=(challenge>>3)&3，计算初值 initial=((uint32_t)(challenge>>6) ^ (uint32_t)(challenge>>32) ^ 0x834c)，从数组 index 起遍历 steps+1 个节点，依次更新

result=5*(result+node.value)^node.tag，并记录末次 tag。这样在只允许修改 defuser.c 的条件下，可用 p5_nodes[index+i] 替代指针追踪。

```c
//这是对phase5代码的改进，将节点整理为数组
typedef struct {
    uint32_t value;
    uint32_t tag;
} P5Node;

static const P5Node p5_nodes[] = {
    {0xe387, 0x0222},
    {0xab32, 0x0608},
    {0x0596, 0x0a58},
    {0x1a18, 0x0411},
    {0xc4a3, 0x007e},
    {0x408c, 0x0827},
    {0x0c6e, 0x0559},
    {0xbf54, 0x0372},
    {0x22e1, 0x0733},
    {0x6b22, 0x0976},
    {0x3713, 0x0b43},
    {0x0f9e, 0x0154}
};
int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    (void)challenge;
    // TODO: 根据 phase_05 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    uint32_t index = (uint32_t)(challenge & 7);
    uint32_t steps = (uint32_t)((challenge >> 3) & 3);
    uint32_t result =
        (uint32_t)(challenge >> 6)
        ^ (uint32_t)(challenge >> 32)
        ^ 0x834c;
    uint32_t tag = 0;
    for (uint32_t i = 0; i <= steps; i++) {
        P5Node node = p5_nodes[index + i];
        result = 5 * (result + node.value) ^ node.tag;
        tag = node.tag;
    }
    values[0] = result;
    values[1] = tag;
    return write_answer(out, cap, values, count);
}
```

图 3 Phase 5 将链表逻辑顺序恢复为静态节点数组的最终实现

3.6 Phase 6：函数指针表与两次间接调用

p6_dispatch 将 challenge 分解为一个 32 位计算参数，以及两个函数表索引：index1=challenge&7，index2=(challenge>>3)&7。汇编通过 call *0x405e00(,%rax,8) 首次调用，再以函数表索引计算结果进行第二次 call *0x405e00(%rbx)。两次之间对返回值异或 0xedf2，末尾加 0x913a。

GDB 读取的 p6_ops 实际排列为 [op_4, op_0, op_5, op_0, op_5, op_1, op_2, op_3]。六种函数包括普通异或 / 加法 / 移位，以及 rol 1 和 ror 5 两种 32 位循环位移。循环位移需要把移出的位重新送回另一端，不能用普通移位代替。实现时定义函数映射并严格按照表索引组合两次运算。seed 42、20261027 与十个公开用例均通过。

```
input = (uint32_t)((challenge >> 6) ^ (challenge >> 32));
t = op_table[challenge & 7](input);
```

```
t ^= 0xedf2;
answer = op_table[(challenge >> 3) & 7](t) + 0x913a;
```

3.7 Honor：固定偏移的字符串输入与运行时排错

隐藏关卡需要先通过全部六关。public_test.py 先运行 defuser 产生六行前置答案，再调用 defuser --secret 产生第七行，并将七行送入 Bomb。secret_gate 进一步调用 secret_phase；后者使用 challenge 和 transcript_context 得到的 context 检查三个无符号整数。最初通过 secret_mix 的 rol 5、+0x2d05、^0xf2ea，以及其它寄存器表达式恢复了三个整数，但公开测试始终为 0/10 wrong_answer。

```
a = (uint32_t)(context & 3);
x = (uint32_t)(challenge >> 32) ^ (uint32_t)challenge;
y = (uint32_t)(context >> 32) ^ (uint32_t)context;
mix(z) = (rol32(z,5) + 0x2d05) ^ 0xf2ea;
b = mix(x ^ y) ^ (a*0x2d05) ^ (uint32_t)(context >> 16);
c = b ^ (y + a + 0xf2ea + (x >> 5));
```

排错先后检查了三种假设：①末尾空格是否被读取函数裁剪；② valid_u32_fields 是否允许空格和 uint32_t 范围；③ defuser 与 Bomb 的 context 是否相同。对 seed 42 构造完整七行输入后，在 secret_gate 入口下断点，GDB 读取 challenge=0xccf635ee9e9e2fa4、context=0xeae75891d43125e5 和输入“OPEN ...”。根据这两个参数独立计算的三整数与 defuser 输出一致，说明公式和 context 不是根因。

进一步把断点放在 secret_phase 调用返回地址 0x402779，程序却在未触发断点前失败。这促使重新检查 secret_gate 的栈偏移：复制目标是 rbp-0xd0；OPEN 常量 0x4e45504f 实际从 rbp-0xb0 比较；整数子串作为参数传给 secret_phase 的地址为 rbp-0xac。这三者相差恰好 32、36 字节，说明 OPEN 并非位于原始字符串起点。修正为“32 个前导空格 + OPEN + 三个整数”，就完全通过测试。

```
===== Honor: Buffer Offset Analysis =====
00000000004026f0 <secret_gate>:
  40270b:       4c 8d a5 30 ff ff ff    lea    -0xd0(%rbp),%r12
  402739:       76 15                   jbe    402750 <secret_gate+0x60>
  402756:       e8 75 ff ff ff          call   4026d0 <secret_copy>
  40275b:       81 bd 50 ff ff ff 4f    cmpl   $0x4e45504f,-0xb0(%rbp)
  402765:       75 d4                   jne    40273b <secret_gate+0x4b>
  40276d:       48 8d bd 54 ff ff ff    lea    -0xac(%rbp),%rdi
  402774:       e8 97 fe ff ff          call   402610 <secret_phase>

===== Calculated Offsets =====
Buffer start : rbp - 0xd0
OPEN position: rbp - 0xb0 (offset 32)
Numbers start: rbp - 0xac (offset 36)
```

图 4 Honor 栈缓冲区的关键汇编证据：起始地址、OPEN 比较位置、整数读取位置

这是全实验最典型的纠错案例：数值计算正确不等于整个输入格式正确；必须把内存访问相对同一缓冲区基址计算，不能从单条 cmpl 指令孤立判断字符串偏移。同时，运行时“断点没有命中”本身就是有效的控制流证据。

四、运行验证与结果

完成全部实现后执行 python3 public_test.py，普通关卡 60/60、Honor 10/10。为满足实验报告对非公开 seed 的要求，另使用不在 tests/public.json 的 seed 20261028，先生成六行普通答案，再追加 Honor 行，将完整文件重定向给 ./bomb --seed 20261028。运行日志中 Phase 1—6 及 Secret phase 均显示 defused，且未出现 BOOM。

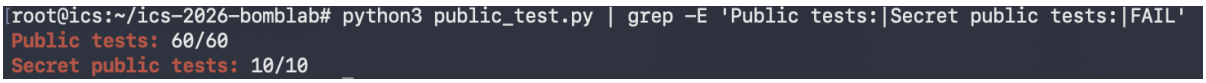


图 5 课程公开测试：普通关卡 60/60、Honor 10/10

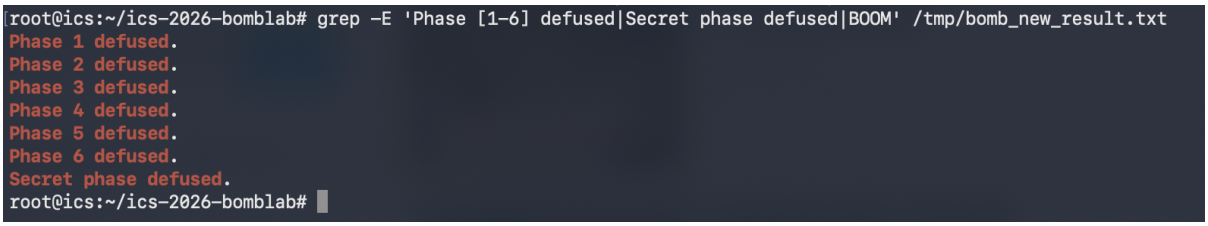


图 6 非公开 seed 20261028 的七关完整成功记录（对应日志 /tmp/bomb_new_result.txt）

验证范围	测试方式	结果
Phase 1—6	public_test.py：10 个公开 seed × 6 关	60/60 PASS
Honor	public_test.py --secret：10 个公开 seed	10/10 PASS
Phase 1—6 + Honor	seed 20261028，七行完整输入	七关均 defused

另附的历史图像包括 Phase 1 源码与 Phase 2 通关画面（图 7），以及最终 Honor 故事结尾（图 8），作为开发过程与完整流程成功的补充材料。

```
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
    (void)challenge;
    // TODO: 根据 phase_01 的汇编，用 challenge 算出口令中的各个整数，依次存入 values，并把 count 改为整数个数。
    uint32_t high = (uint32_t)(challenge >> 32);

    uint32_t middle = (uint32_t)(challenge >> 16);

    uint32_t low = (uint32_t)challenge;

    uint32_t result1 = (middle + 0xc699) ^ high;

    uint32_t result2 = (3 * low) ^ (high + 0x71b5);

    uint32_t answer = 5 * (result2 - result1) + 0x96fa;

    values[0] = answer;

    count = 1;

   return write_answer(out, cap, values, count);
}

int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 6;
root@ics:~/ics-2026-bomblab# make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
root@ics:~/ics-2026-bomblab# ./defuser --seed 20261027 --phase 2
30 150 750 3760 18810 94070
root@ics:~/ics-2026-bomblab# ./defuser --seed 42 --phase 2 | ./bomb --seed 42 --phase 2
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

图 7 Phase 1 的 C 语言实现和 Phase 2 的本地运行验证（过程记录）

```
  — sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。

  这里的东西我会留着。你在北方有了新想法，也可以回来添。
  我暂时不走，这边的事刚有起色，想再试一阵。
  以后说不定会改主意，到时候告诉你。你也一样。
  别因为申请时说得很坚定，遇到不喜欢的事就不好意思讲。

  第一场雪记得拍。我想看看你一直说的北方。

  — 许遥

后面是这些年断断续续的更新。
一张北方的雪照，一次换了方向的项目，一段拖了很久才修好的代码。
sky 去了北方，许遥留在这里。他们各自忙起来，
回复有时隔了好几天，也会聊新认识的朋友。
有一阵，许遥没再继续原来的项目。后来 sky 也换了工作。
这座城倒还在。谁回来，就顺手添一点东西。

观测台的灯是 sky 修的，两把椅子是许遥后来搬来的。
最后一张便签贴在灯座旁边：

  ‘下次回来，先坐一会儿。’

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。
街上亮起今天的新灯。

离开观测台时，你想起自己收藏了很久的一份项目介绍，
把它放进明天的待办。林夏也在自己的本子上记了几行。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

晚安。

— sky

root@ics:~/ics-2026-bomblab#
```

图 8 Honor 通过后输出的完整叙事结尾（成功运行的补充记录）

五、经验总结

第一，逆向阅读首先要厘清控制流与数据流：从返回值比较反推输入格式，顺着寄存器和调用约定建立 C 语言关系。第二，机器级数据结构的“逻辑顺序”可能与内存排列不同；Phase 5 用实际 next 指针恢复链表，再重构静态数组，使算法更易检查。第三，编译成功、单次运行成功和多 seed 验证是不同层级的证据；修改 C 源码后必须重新 make，避免旧可执行文件造成误判。第四，Honor 表明定位 bug 不能只盯公式；运行时断点、长度检查及栈偏移的联合观察最终揭示了失败原因。

六、工具使用与资料说明

资料依据：课程提供的 BombLab README、Makefile、tutor 教学文件、个人 bomb 的 objdump 输出、GDB 内存与寄存器读取，以及 public_test.py 验证脚本。使用的开发工具包括 gcc/make（构建）、objdump/nm（符号与反汇编）、gdb（跳转表、链表、函数指针和动态断点）、Python（批量测试）、Git（版本管理）。

AI 辅助说明：实验过程中使用 ChatGPT 解释 x86-64 汇编与 C 代码的对应关系、、设计 GDB 断点及测试步骤。与 AI 多次交互排错的部分尤其涉及 Phase 4 的函数结构错误、Phase 5 的节点重排思路。

七、想对助教说的话

难度确实有点大。。。花费了巨量的时间，不断与 ai 切磋调试 TAT，希望对于双学位班的同学们能够控制难度

没想到有一天能在 lab 里看到情节！看雪的情节挺浪漫的，就是总体来说故事不是很长，貌似没能搭建起一个比较完整的世界观？（）

希望助教能够保证虚拟机与 GitHub 连接的稳定，本次 lab 曾在三个不同的时间段尝试克隆都没有成功 QAQ